WEEK 1 · DAY 5 · OVERVIEW

Case Study

The Market Dataset



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Definition

Week 1 ends today, and weeks 2 and 3 start from one dataset. First the pulls: a morning of reruns must not send FRED hundreds of requests, so keep a copy of each series for the day, and prove the copy gives back every digit. ICE and Moody's series must never be stored: make the code refuse them, not a habit. Show me the same curve comes from FRED through that copy, equal to our file, and tag the week. Then build the daily dataset, five Treasury tenors and two credit spreads, one row per market day, and prove it clean with every check of the week. Describe the data: no view on any rate or spread.

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

Your Route Through Day 5

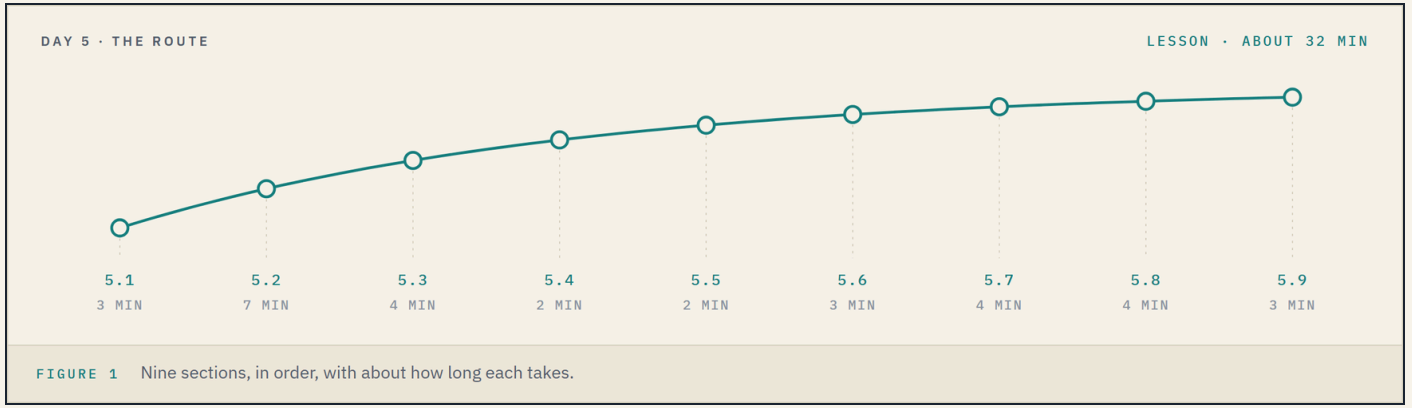


FIGURE 1 Nine sections, in order, with about how long each takes.

	SECTION	ABOUT
5.1	Eleven Requests a Run	3 min
5.2	A Copy That Gives Back Every Digit: `write_cache` and `read_cache`	7 min
5.3	`cached_fred_series`, and Why It Refuses Licensed Series	4 min
5.4	Tag the Week	2 min
5.5	Live: The Curve From FRED, Through the Cache	2 min

	SECTION	ABOUT
5.6	ICE and Moody's on FRED: Metadata Only	3 min
5.7	The Curve, Checked	4 min
5.8	The Spreads, Joined	4 min
5.9	The Dataset	3 min

Time. About 32 minutes for the lesson, then about 12 for the four exercises.

5.1 Eleven Requests a Run

WHAT A MORNING COSTS

```
importlib.reload(md)
requests_per_run = len(md.FRED_TREASURY)
print(f'Twelve runs: {12 * requests_per_run} requests without a cache, '
      f'{requests_per_run} with one')
```

```
Twelve runs: 132 requests without a cache, 11 with one
```

THE MODULE AS IT WAS LEFT

```
.................                                                [100%]
17 passed in 0.50s
```

Without a cache every rerun costs 11 more requests; with one, the whole day costs 11.

5.2 A Copy That Gives Back Every Digit: write_cache and read_cache

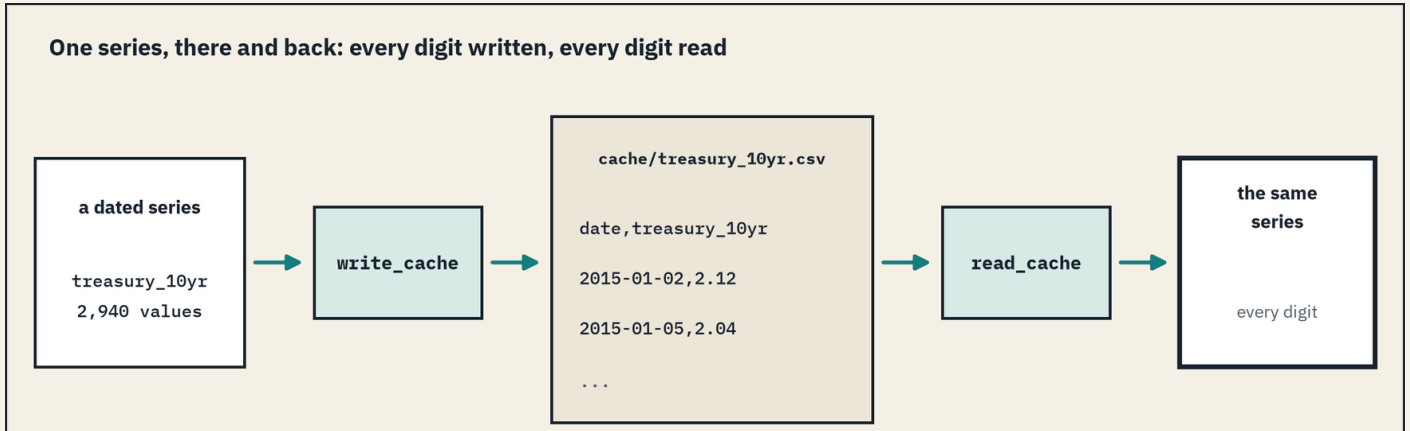


FIGURE 2 write_cache saves the 10 Yr to treasury_10yr.csv and read_cache returns the same series, every digit.

THE FILE IT WRITES

```
date,treasury_10yr
2015-01-02,2.12
2015-01-05,2.04
```

READ IT BACK

```
2,940 of 2,940 values back, every one exact
```

Every digit comes back because `read_cache` asks for every digit. Next, the function that decides whether to ask FRED at all.

5.3 cached_fred_series, and Why It Refuses Licensed Series

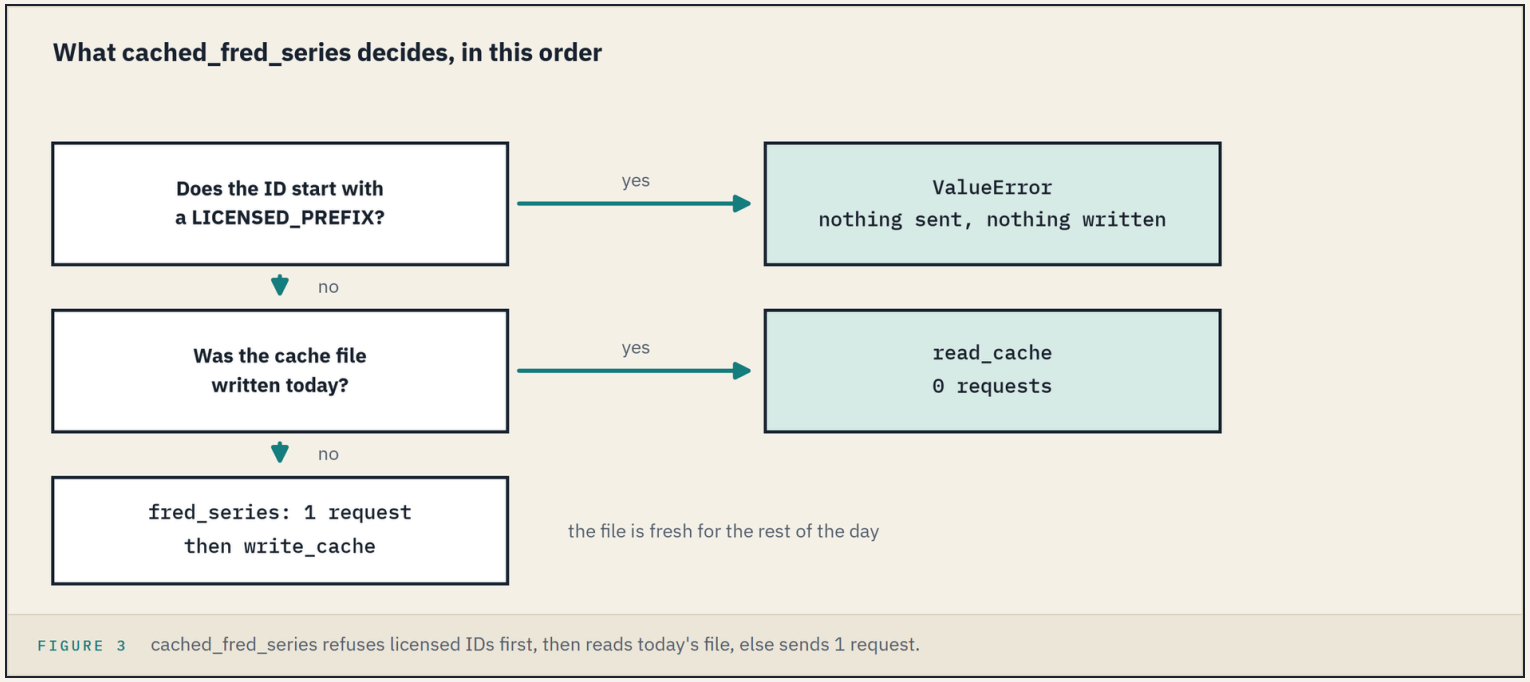


FIGURE 3 cached_fred_series refuses licensed IDs first, then reads today's file, else sends 1 request.

WHICH IDS MAY BE CACHED

```
DGS10          may be cached
DGS2           may be cached
BAMLC0A0CM     refused: licensed
BAMLH0A0HYM2   refused: licensed
BAA10Y         refused: licensed
DBAA           refused: licensed
```

No request, no file: a check that stops beats a habit. Week 1's module is complete.

5.4 Tag the Week

NAME THE WEEK

```
!git -C "{work}" tag week1
!git -C "{work}" log --oneline --decorate
```

```
ba73980 (HEAD -> main, tag: week1) Notebook 05: write_cache, read_cache, and cached_fred_series
9f01b1c Notebook 04: last_per_period and value_as_of
a439502 Notebook 03: missing_weekdays, align, and stale_runs
ceea769 Notebook 02: fred_params, parse_fred_observations, and fred_series
a28686d Notebook 01: get_secret, a .gitignore, and .env.example
```

WHAT GIT SEES

```
 M test_marketdata.py
.gitignore:4:cache/     cache/treasury_10yr.csv
```

`week1` **names the module as the week ends: three functions, three tests, and a cache Git never sees. Now the copy on real data.**

5.5 Live: The Curve From FRED, Through the Cache

THE LIVE PULL

```
    for live_id, live_tenor in md.FRED_TREASURY.items():
        live_series = md.cached_fred_series(live_id, fred_key, live_cache, start=live_start)
        live_gap = (live_series[live_common] - treasury.loc[live_common, live_tenor]).abs().max()
        assert len(live_common) > 0 and live_gap <= 1e-9, f'{live_id} differs from the file'
    for live_id in md.FRED_TREASURY:
        live_again = md.cached_fred_series(live_id, fred_key, live_cache, start=live_start)
```

WITH YOUR KEY

Eleven lines, each with a largest gap of `0.0e+00` percent: FRED's `DGS` series hold the Treasury file's published values.

THEN AGAIN

The second pass reads eleven files and sends nothing.

The cache works on the real thing. FRED carries credit series too.

5.6 ICE and Moody's on FRED: Metadata Only

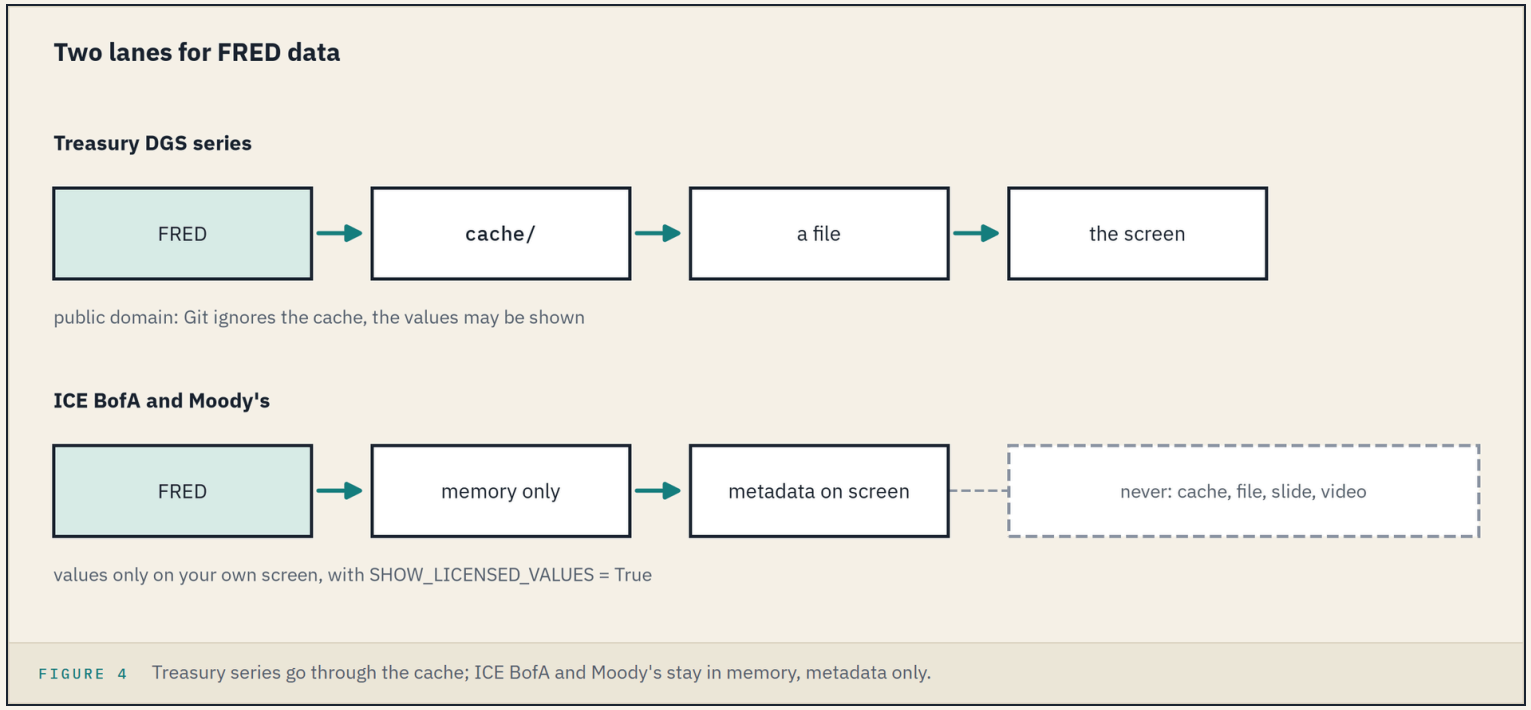


FIGURE 4 Treasury series go through the cache; ICE BofA and Moody's stay in memory, metadata only.

WITH YOUR KEY

One line per series: about three years of daily ICE rows, with weekend rows at month ends, and a longer `BAA10Y`; then the cache line.

Licensed values never reach a cache, a file, a slide, or a video, so the dataset's credit columns are synthetic, and labelled so. Now the case study.

5.7 The Curve, Checked

DATES: UNIQUE, ASCENDING, WEEKDAYS

```
assert treasury.index.is_unique and treasury.index.is_monotonic_increasing, 'dates out of order'
weekend_rows = (treasury.index.dayofweek >= 5).sum()
assert weekend_rows == 0, 'a weekend date'
print(f'{len(treasury):,} dates, unique and ascending, {weekend_rows} on a weekend')
```

```
2,940 dates, unique and ascending, 0 on a weekend
```

EXCEL SIDE BY SIDE

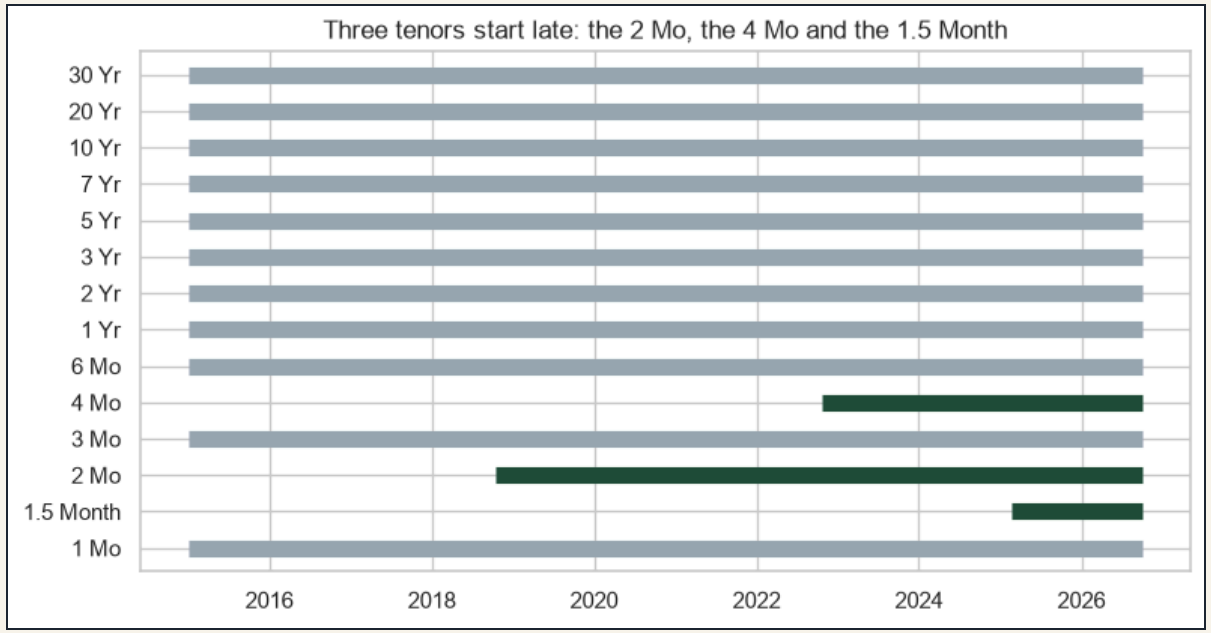
JOB	EXCEL
Weekend dates	`=SUMPRODUCT(--(WEEKDAY(A2:A2941,2)>5))`
Repeated dates	`=ROWS(A2:A2941)-ROWS(UNIQUE(A2:A2941))`
Dates out of order	`=SUMPRODUCT(--(A3:A2941<=A2:A2940))`

THE MARKET'S OWN CALENDAR

```
print(f'{len(missing)} weekdays with no row, the last {missing[-1].date()}')
```

```
126 weekdays with no row, the last 2026-09-07
```

Three Tenors Start Late



THREE TENORS START LATE

```
1.5 Month from 2025-02-18: 2,532 blank rows before, 0 after
2 Mo      from 2018-10-16: 949 blank rows before, 0 after
4 Mo      from 2022-10-19: 1,951 blank rows before, 0 after
```

EXCEL SIDE BY SIDE

JOB	EXCEL
Blank cells of the 4 Mo, column F	`=COUNTBLANK(F2:F2941)`
The 4 Mo's first date	`=XLOOKUP(TRUE,F2:F2941<>"",A2:A2941)`

A blank before a tenor starts is not a missing value: nothing was published. The dataset takes five tenors that run the whole file.

5.8 The Spreads, Joined

One row per market day: both files skip the same days

	Fri 04 Sep	Sat 05 Sep	Sun 06 Sep	Mon 07 Sep	Tue 08 Sep	Wed 09 Sep	Thu 10 Sep	Fri 11 Sep
curve, 10 Yr	4.78	no row	no row	no row	4.80	4.83	4.95	4.96
IG, synthetic	94.5	no row	no row	no row	93.3	93.3	93.6	91.5

Labor Day

Inner and outer joins keep the same five rows: neither file has a date the other lacks.

FIGURE 5 Labor Day week 2026: the 10 Yr and IG spread share their rows, so inner and outer joins keep the same ones.

STACK AND JOIN

```
stacked 2,940 rows, equal to the file: True
merged 2,940 rows, 17 columns with the date
```

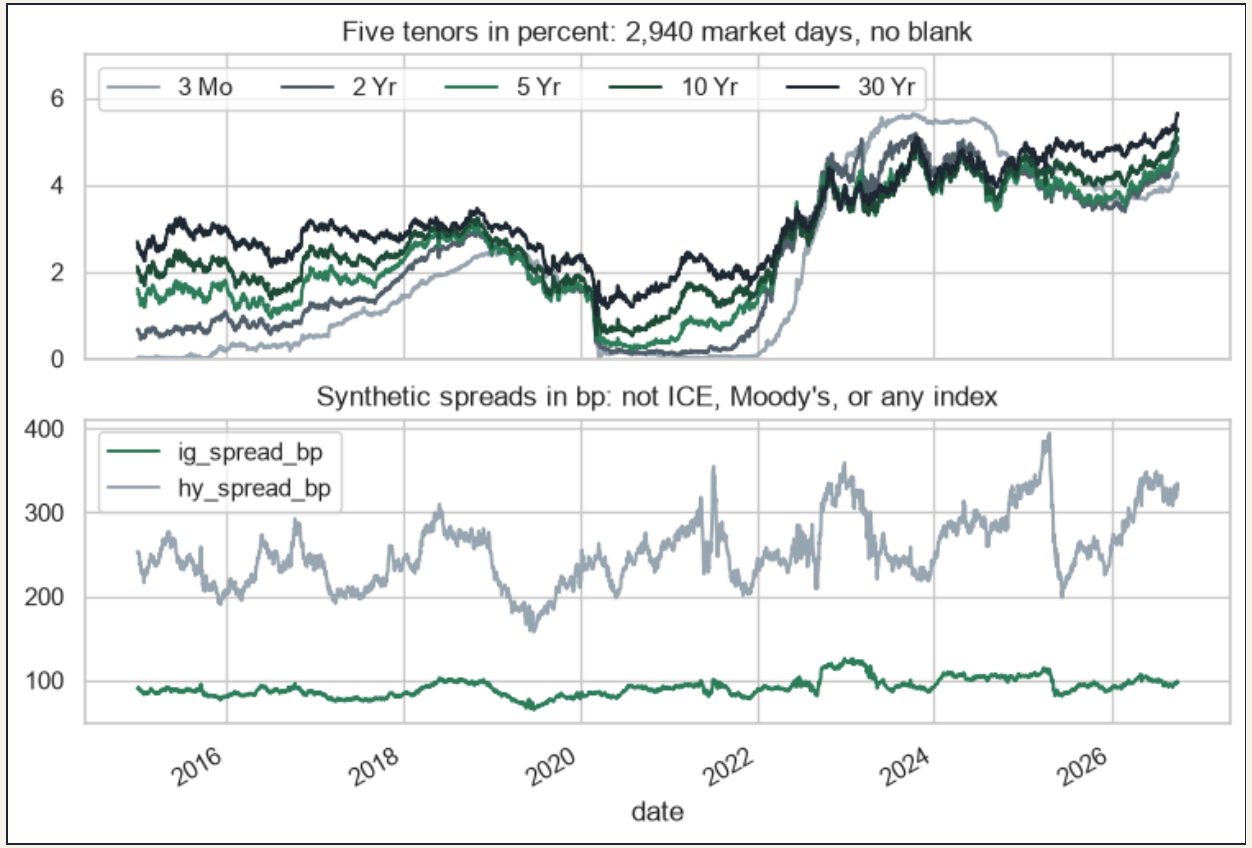
INNER AND OUTER

```
inner 2,940 rows, outer 2,940 rows, 0 blank
```

EXCEL SIDE BY SIDE

JOB	EXCEL
Stack 2,750 and 190 rows	Append: `Table.Combine({First, Second})`
Join on the date	Merge: `Table.NestedJoin(... JoinKind.LeftOuter)`, then `Table.ExpandTableColumn`
A spread beside each date	`=XLOOKUP($A2,Spreads!$A$2:$A$2941,Spreads!$B$2:$B$2941)`, filled down

5.9 The Dataset



BUILD IT

```
dataset = md.align(columns, how='inner')
print(f'{len(dataset):,} rows, {dataset.shape[1]} columns')
```

```
2,940 rows, 7 columns
```

EXCEL SIDE BY SIDE

JOB	EXCEL
Blank looked-up spreads, column P	`=COUNTBLANK(P2:P2941)`

2,940 market days, seven columns, no blank, no weekend, no repeated date, no stale spread: the dataset weeks 2 and 3 start from. The note is answered.

You Can Now

- Say what a cache saves: FRED's limit, time, and a pull when FRED is down.
- Cache a series and read it back exactly, with **`float_precision="round_trip"`**.
- Use **`cached_fred_series`**, which pulls at most once a day and refuses a licensed series before sending anything.
- Keep ICE's and Moody's values off every saved or shared screen.
- Build a daily market dataset and prove it clean with every check of week 1.
- Tag the end of a week in Git and find the tag with **`git log --decorate`**.

NEXT
WEEK 2 · DAY 1

Changes in bp and Percent

How far each series moved, in its own unit: yields and spreads in bp, prices in percent, over a day, a week, a month and the year, tied to Excel.